but it also has some drawbacks and complexity:

- Configuration is split between two files.
- The separation between Program.cs and `Startup` is somewhat arbitrary.
- The generic `IHostBuilder` exposes newcomers to legacy decisions.
- The lambda-based configuration can be hard to follow and reason about.
- The pattern-based conventions of `Startup` may be hard to discover.
- Tooling historically relies on your defining a `CreateHostBuilder` method in Program.cs.

I'll address each of these problems in turn and afterward discuss how `WebApplication` attempted to improve the situation.

Points 1 and 2 in the preceding list deal with the separation between Program.cs and `Startup`. As you saw in section 30.1, theoretically the intention is that Program.cs defines the host and rarely changes, whereas `Startup` defines the app features (services, middleware, and endpoints). This seems like a reasonable decision, but one inevitable downside is that you need to flick back and forth between at least two files to understand all your bootstrapping code.

On top of that, you don't necessarily need to stick to these conventions. You can register services in Program.cs by calling `HostBuilder.ConfigureServices()`, for example, or register middleware using `WebHostBuilder.Configure()`. This